SMART ACADEMIC ADVISOR

Graduation Project Proposal

Egyptian Russian University
Faculty of Artificial Intelligence

Submitted by:

Name	**ID**	**Supervised by:**
Amr Hamdy	225182	Dr. Sameh Zaref
Youssef Emad	225241	
Omar Tokal	225238	**Teaching Assistant:**
Zeyad Mostafa	225070	Eng. Mariam Hagag

Project Description

The Smart Academic Advisor is an intelligent academic guidance system designed to replicate and enhance the role of a human academic advisor. The system integrates Artificial Intelligence (AI) with Retrieval-Augmented Generation (RAG), Knowledge Graph reasoning, and a Large Language Model (LLM) to deliver a personalized and adaptive advising experience for university students. Through a mobile application built with Flutter, connected to a Supabase backend and a FastAPI-based AI engine, students can interact naturally with the system to receive course recommendations, prerequisite guidance, and academic planning support. The chatbot assistant interprets student queries, retrieves relevant academic data from university documents, and applies semantic reasoning across a knowledge graph of courses and prerequisites. This solution simplifies academic decision-making, enhances communication between students and the advising system, and promotes proactive educational planning. Ultimately, the Smart Academic Advisor improves accessibility, efficiency, and overall satisfaction with the advising process, offering a reliable digital alternative to traditional manual advising methods.

Problem Statement

In traditional academic advising systems, students often face limited access to advisors, delayed responses, and inconsistent guidance. These challenges can lead to poor course selection, unbalanced academic loads, and difficulty in tracking prerequisite requirements. Furthermore, human advisors are constrained by time and availability, making it difficult to provide continuous and personalized support to every student. The Smart Academic Advisor addresses these limitations by introducing an AI-driven virtual advisor capable of providing real-time, personalized, and data-driven recommendations. By integrating RAG for information retrieval, a Knowledge Graph for semantic understanding, and an LLM for natural conversation, the system delivers accurate, context-aware, and student-specific guidance. This innovation not only reduces the workload on human advisors but also ensures that every student receives reliable academic support whenever needed.

Objectives

1. To create an intelligent academic advising system that improves the efficiency and accessibility of the advising process.
2. To support students in making informed academic decisions through personalized and data-driven guidance.
3. To reduce the workload on human advisors by automating routine academic advising tasks.
4. To enhance students' overall academic experience by providing timely and reliable assistance.
5. To utilize artificial intelligence to deliver accurate, adaptive, and effective academic support within a digital environment.

Technical Phases

Question Answering Chatbot Development Timeline (15 October – 30 December)

Phase 1: Data Collection & Preparation (15 Oct – 31 Oct)

Goal: Gather and preprocess all academic documents and data for RAG and Knowledge Graph.

Tasks:

- Collect course catalogs, rules, and syllabi PDFs.
- Extract and clean text using PyPDF2 or pdfplumber.
- Design the Knowledge Graph schema (courses, majors, prerequisites).
- Prepare structured course datasets (CSV/JSON).

Deliverable: Clean dataset and defined KG schema ready for system development.

Phase 2: Knowledge Graph Construction (1 Nov – 15 Nov)

Goal: Build and populate the Knowledge Graph database.

Tasks:

- Create nodes and relationships in Neo4j (e.g., Course, Prerequisite, Major).
- Import structured data into Neo4j.
- Test Cypher queries (e.g., 'List all prerequisites for AI201').

- Connect graph database with Python backend using the neo4j-driver.

Deliverable: Fully functional Knowledge Graph with verified relationships.

Phase 3: RAG (Retrieval-Augmented Generation) Development (16 Nov – 30 Nov)

Goal: Implement document retrieval and embedding search.

Tasks:

- Chunk academic PDFs using LangChain text splitter.
- Generate text embeddings using text-embedding-3-large or sentence-transformers.
- Store and index chunks in FAISS or Supabase vector database.
- Test top-k retrieval accuracy.

Deliverable: RAG system able to retrieve relevant content from documents.

Phase 4: Hybrid QA Pipeline Integration (1 Dec – 15 Dec)

Goal: Combine retrieval and reasoning to generate context-aware answers.

Tasks:

- Build hybrid function combining RAG and Knowledge Graph outputs.
- Integrate LLM (GPT-4 or Claude) for reasoning and final answer generation.
- Create prompt templates for academic tone and structured responses.
- Implement backend orchestration pipeline.

Deliverable: Hybrid QA chatbot backend producing accurate, context-rich answers.

Phase 5: Testing & Optimization (16 Dec – 30 Dec)

Goal: Evaluate performance and refine system reliability.

Tasks:

- Conduct internal testing for different academic question types.
- Measure accuracy, latency, and consistency of answers.
- Optimize retrieval and graph queries.
- Prepare documentation and connect API endpoint /ask for Flutter integration.

Deliverable: Stable and optimized Question Answering system ready for frontend integration.

Flutter Mobile App Development Timeline (15 February – 30 May)

Phase 1: Project Foundation (15 Feb – 7 Mar)

Goal: Establish the Flutter project environment and design foundation.

Tasks:

- Configure Flutter environment and dependencies.
- Set up version control (GitHub, branching workflow).
- Define app architecture (Flutter ↔ FastAPI ↔ Supabase).
- Design UI/UX layouts and chat interface wireframes.
- Review and approve requirements with supervisor.

Deliverable: Approved design and initialized Flutter project.

Phase 2: Backend & Database Connection (8 Mar – 28 Mar)

Goal: Connect mobile app with backend and Supabase.

Tasks:

- Integrate Supabase authentication for student login.
- Connect app to FastAPI endpoints.
- Implement real-time communication with /ask chatbot endpoint.
- Design models for student profiles, chat history, and recommendations.

Deliverable: Connected backend and working user authentication.

Phase 3: Core App Development (29 Mar – 25 Apr)

Goal: Develop main app screens and essential features.

Tasks:

- Build key screens (dashboard, profile, courses, academic summary).
- Manage app state using Riverpod/Bloc.
- Handle secure API calls and data fetching.
- Ensure smooth navigation and responsiveness.

Deliverable: Functional app with integrated backend data.

Phase 4: Chatbot System Integration (26 Apr – 9 May)

Goal: Integrate the Question Answering Chatbot into the mobile app.

Tasks:

- Implement the chatbot interface (real-time message stream).
- Connect with backend QA API (RAG + KG + LLM).
- Display contextual and human-like responses.
- Enable chat history storage in Supabase.

Deliverable: Working chatbot integrated in the app with backend responses.

Phase 5: Advanced Features & Analytics (10 May – 24 May)

Goal: Enhance personalization and intelligence in the chatbot experience.

Tasks:

- Add personalized advising based on student ID and course history.
- Integrate analytics dashboard for chat tracking.
- Improve UI animations and UX flow.
- Optimize performance and reduce API latency.

Deliverable: Advanced, intelligent, and user-friendly chatbot experience.

Phase 6: Testing & Finalization (25 May – 30 May)

Goal: Validate system stability and finalize project for submission.

Tasks:

- Conduct full functional and user acceptance testing.
- Fix bugs, optimize performance, and ensure security.
- Prepare demo video, final documentation, and slides.
- Conduct final supervisor review.

Deliverable: Stable and complete app ready for project submission and defense.